

תרגול 9 - Node.js

כל הזכויות שמורות לקרן יעקב 2025



1. התקנות

התקנת Node.js

Run JavaScript Everywhere

Node.js® is a free, open-source, cross-platform JavaScript runtime environment that lets developers create servers, web apps, command line tools and scripts.

Download Node.js (LTS) 

Downloads Node.js **v20.14.0**¹ with long-term support.
Node.js can also be installed via package managers.

קישור: <https://nodejs.org/en>

נוריד את קובץ ההתקנה LTS (תמיכה לטווח ארוך) - גרסה 20.14.0

התקינו את Node.js במחשב שלכם

ה-mac אמור להיות מותקן ביחד

נבדוק בעמוד הבא

על OS MAC יתכן וצריך להתקין גם XCode מאחר ו-Node מניח שיש לכם קומפיילר של ++C להרצת הפקודות של Node.

אין צורך כרגע בקינפוגים ובהורדות נוספות

בדיקת התקנת Node.js

```
Command Prompt
C:\Users\KerenY>node -v
v20.14.0
C:\Users\KerenY>npm -v
10.8.1
```

● בטרמינל/CMS, כרגע לא משנה באיזו תיקיה:

○ כתבו `node -v`

○ כתבו `npm -v`

○ אם מוצגים מספרי גירסאות, כמו בתמונת מסך מצד שמאל, ההתקנה בסדר!

○ אם `node` לא מותקן - חזרו לעמוד הקודם והתקינו

○ אם `npm` לא מותקן - ב-CMD רשמו את הפקודה:

`npm install -g npm@latest`

בדיקת התקנת Node.js – המשך

- פתחו קובץ js בשם node1.js וכתבו בקובץ את השורה הבאה:

```
console.log("Hello world!");
```

A screenshot of a code editor window. The title bar shows a file named 'node1.js' with a yellow icon and a close button. The editor content shows a single line of code: `console.log("Hello world!");` on line 1. The background is dark, and the text is light.

- בטרמינל הגיעו לתיקיה בה שמור הקובץ וכתבו:

```
$node node1.js
```

פקודה זו מריצה את ה-js ולכן אם הכל הותקן והופעל כשורה נראה בטרמינל:

A screenshot of a terminal window. The title bar has tabs for 'TERMINAL', 'PROBLEMS', 'OUTPUT', 'DEBUG CONSOLE', 'PORTS', and 'GITLENS'. The terminal content shows a command prompt 'PS C:\Users\KerenY\Downloads\Web 2024\Shenkar2024Keren\9.2-Nodejs-tar>' followed by the command 'node node1.js' and its output 'Hello world!'.

- את ה-\$ לא מקלידים, זה סימן שיש להקליד את הפקודה בטרמינל/CMD

פוסטמן

צרו חשבון באתר [/https://www.postman.com](https://www.postman.com)

ניתן להשתמש בגירסת הווב, אבל מומלץ להוריד מכאן: [/https://www.postman.com/downloads](https://www.postman.com/downloads)

The Postman app

Download the app to get started with the Postman API Platform.

Windows 64-bit

By downloading and using Postman, I agree to the [Privacy Policy](#) and [Terms](#).

[Release Notes](#) →

Not your OS? Download for Mac ([Intel Chip](#), [Apple Chip](#)) or Linux ([x64](#), [arm64](#))

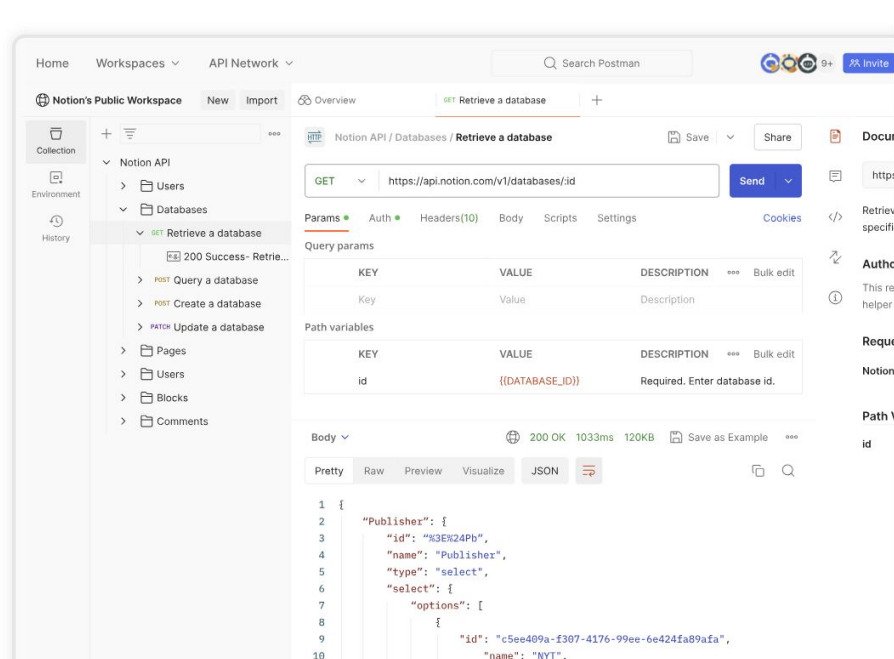
Postman on the web

Access the Postman API Platform through your web browser. Create a free account, and you're in.

Try the Web Version

Postman Enterprise

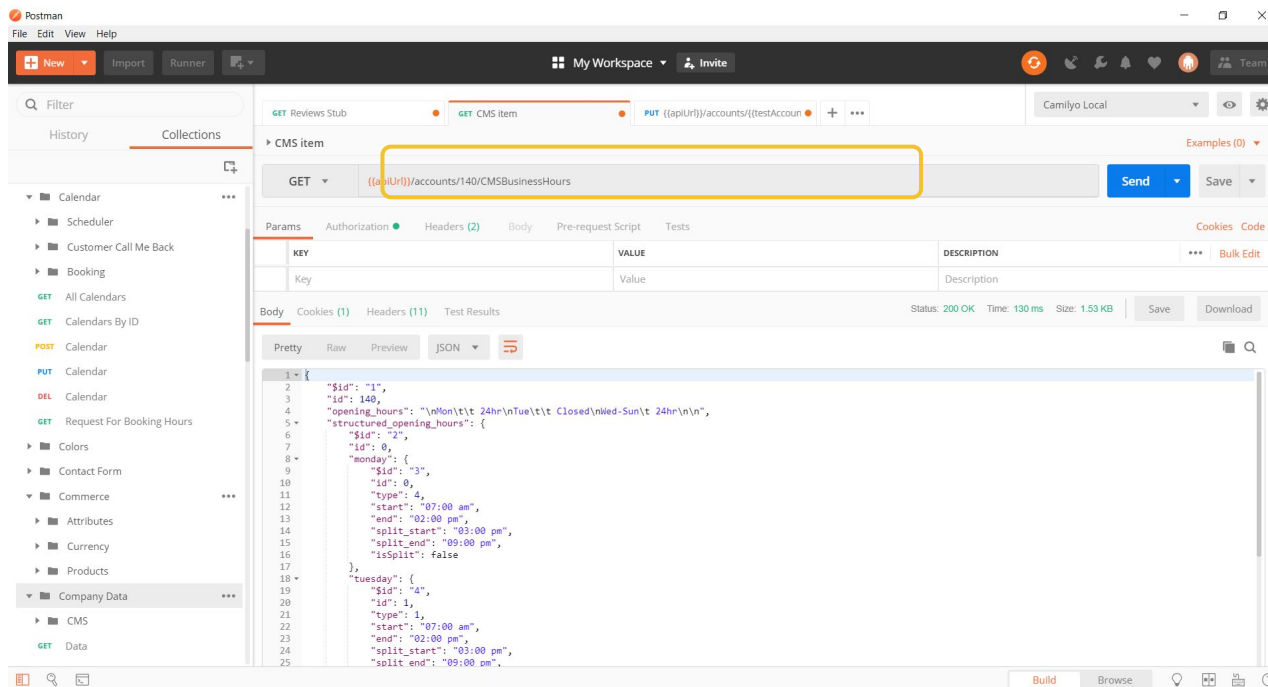
Postman Enterprise is designed for organizations who



פוסטמן

כשאנחנו כותבים את צד השרת – נרצה לבדוק את תקינות הקריאות.
נשתמש בפוסטמן כדי לעשות קריאות לשרת, במקום צד הלקוח שיתוכנת בהמשך.

דוגמה לשימוש:





Node – Modules .2

תרגיל 1 – Built-in Node global modules

צרו קובץ js וכתבו בו:

```
node3.js > ...
1  const os = require("os");
2  const msg = `Js running on process version: ${process.version}, on OS: ${os.type()}`;
3  console.log(msg);
```

נריץ ע"י node, התוצאה:

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS  GITLENS

PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node> node node3.js
Js running on process version: v20.14.0, on OS: Windows_NT
```

בגלל שהקוד אינו רץ בדפדפן אלא ב-command line, הוא רץ בתהליך - process.
Process הוא אובייקט גלובלי המציג מידע על התהליך שרץ.

תרגיל 2 – NPM modules

● ניצור את הקובץ package.json הראשון שלנו

○ צרו ספרייה חדשה

○ שימו בה קובץ index.js, כרגע ריק

○ נתבו אל התיקייה על ידי - cd path \$

○ בכתוב: npm init והוא יתחיל להציג רשימת שאלות

○ נאשר הבל, נגדיר רק את שם ה-author השאר לא קריטי

○ y- npm init יחזיר yes להכל

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS  GITLENS

See `npm help init` for definitive documentation on these fields
and exactly what they do.

Use `npm install <pkg>` afterwards to install a package and
save it as a dependency in the package.json file.

Press ^C at any time to quit.
package name: (init1)
version: (1.0.0)
description:
entry point: (index.js)
test command:
git repository:
keywords:
author: Keren
license: (ISC)
About to write to C:\Users\Keren\Kerens\Shenkar\9-WebService2022A\Code\3-Node\init1\package.json:

{
  "name": "init1",
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "author": "Keren",
  "license": "ISC",
  "description": ""
}

Is this OK? (yes) y
```

```
package.json X
9-WebService2022A > Code > 3-Node > init1 > package.json > ...
1  {
2    "name": "init1",
3    "version": "1.0.0",
4    "main": "index.js",
   Debug
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "author": "Keren",
9    "license": "ISC",
10   "description": ""
11  }
```

תרגיל 2 – NPM modules – המשך

1. \$ npm install express
2. \$ npm install nodemon --save-dev

● נרצה להוסיף שני מודולים לפרוייקט שלנו:

○ מודול אחד ל- dependencies :

○ מודול נוסף ל- dev-dependencies :

● נוצרה ספריית node_modules ליד package.json.

● הקבצים והתיקיות נוצרו בפרויקט שלנו- כל ה dependencies שהמודולים צריכים

```
TERMINAL  PROBLEMS  OUTPUT  DEBUG CONSOLE  PORTS  GITLENS

PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node\init1> npm install express

added 64 packages, and audited 65 packages in 5s

12 packages are looking for funding
  run `npm fund` for details

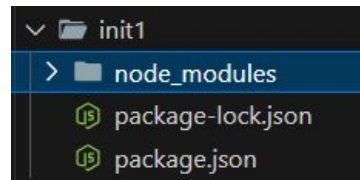
found 0 vulnerabilities
```

```
PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node\init1> npm install nodemon --save-dev

added 29 packages, and audited 94 packages in 4s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

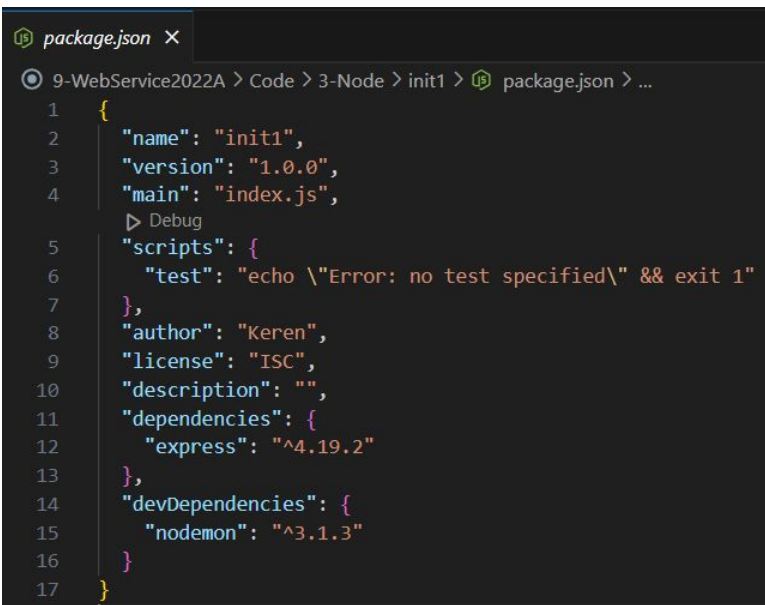


תרגיל 2 – NPM modules – המשך

נפתח את קובץ ה-`package.json` שעוקב אחרי ההתקנות של המודולים שלנו. שימו לב שהוא JSON חוקי ולא אובייקט JS

התווסף לנו מודול נוסף ל-`dependencies` : **express**

התווסף לנו מודול נוסף ל-`dev-dependencies` : **nodemon**



```
package.json X
9-WebService2022A > Code > 3-Node > init1 > package.json > ...
1  {
2    "name": "init1",
3    "version": "1.0.0",
4    "main": "index.js",
5    "scripts": {
6      "test": "echo \"Error: no test specified\" && exit 1"
7    },
8    "author": "Keren",
9    "license": "ISC",
10   "description": "",
11   "dependencies": {
12     "express": "^4.19.2"
13   },
14   "devDependencies": {
15     "nodemon": "^3.1.3"
16   }
17 }
```

תרגיל 2 – NPM modules – המשך

```
5  "scripts": {  
6    "test": "echo \"Error: no test specified\" && exit 1"  
7  },
```

שימו לב לשורות 5-7 : scripts

ניתן להוסיף סקריפטים ש-npm יריץ אוטומטית

לדוגמה אם נרצה שהפקודה `npm start` תריץ את הפקודה `node index.js`

עכשיו נוסיף פקודה נוספת שתריץ את nodemon

nodemon הוא כלי יעיל לפיתוח שעושה watch לקבצים ומריץ את node מחדש בעת שמירת שינויים
לא נצטרך לעצור את השרת ולהפעיל מחדש לאחר כל שינוי שלנו בקוד

נוסיף סקריפט: `"dev": "nodemon index.js"`

בעת הריצו שוב, הפעם הכניסו בטרמינל את הפקודה `npm run dev`

```
License: ISC ;  
PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node\init1> npm run dev
```

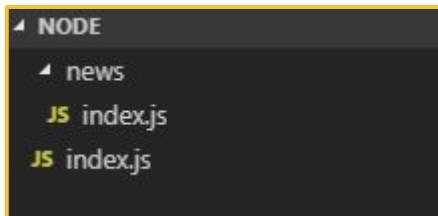
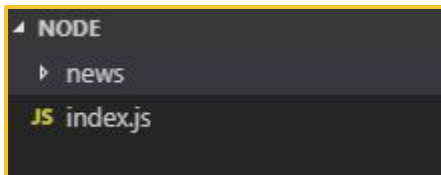
```
> init1@1.0.0 dev  
> nodemon index.js
```

```
[nodemon] 3.1.3  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node index.js`  
[nodemon] clean exit - waiting for changes before restart
```

```
5  "scripts": {  
6    "test": "echo \"Error: no test specified\" && exit 1",  
7    "dev": "nodemon index.js"  
8  },
```

תרגיל 3 – require + module.exports

- ניצור ספרייה חדשה
- בתוכה קובץ index.js – הקובץ המפעיל
- וספרייה news שבתוכה index.js
- זכרו שאין פה את צד האפליקציה אצל המשתמש (רק צד השרת).
לא לבלבל, אנחנו בתוך שירות – web service.



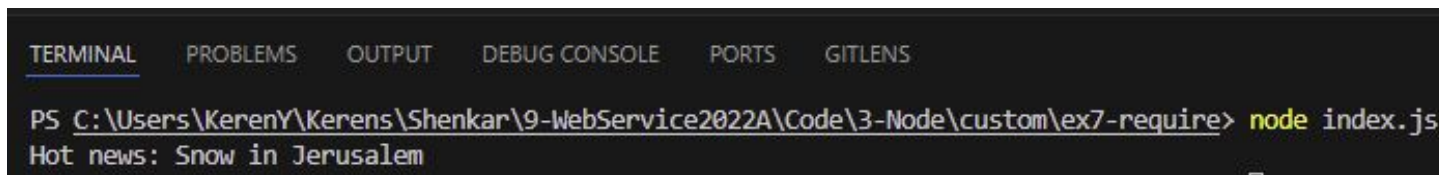
תרגיל 3 – require + module.exports – המשך

- הכניסו את השורות הבאות לקבצים המתאימים
- איזה מה-index.js נריץ? איך אפשר לדעת?



```
JS index.js x
JS index.js ▾ ...
1 const hotNews = require("./news/index"); // With or without .js
2 hotNews("Snow in Jerusalem");

news ▾ JS index.js ▾ ...
1 const prefixMsg = "Hot news: ";
2
3 module.exports = msg => console.log(prefixMsg + msg);
4
```



```
TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE PORTS GITLENS
PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node\custom\ex7-require> node index.js
Hot news: Snow in Jerusalem
```

- נשים לב שיש שימוש ב-module.exports, בתרגיל הבא נראה עוד דרך.

תרגיל 4 – exports

ex4 > JS index.js > ...

```
1 const speedDrive = require("./drive");
2
3 speedDrive.slowly("Go faster, this is too slow");
4 speedDrive.fast("Hey! Slow down! This is too fast");
5
```

ex4 > drive > JS index.js > ...

```
1 const slow = msg => console.log(`Dan said: ${msg}`);
2
3 exports.slowly = slow;
4
5 exports.fast = msg => console.log(`Dan cried: ${msg}`);
```

TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE PORTS GITLENS

```
PS C:\Users\KerenY\Kerens\Shenkar\9-WebService2022A\Code\3-Node\custom\ex8-module> node index.js
Dan said: Go faster, this is too slow
Dan cried: Hey! Slow down! This is too fast
```

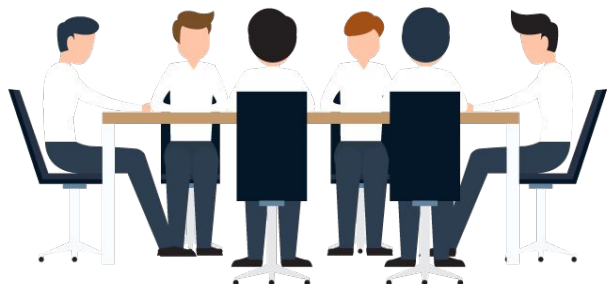
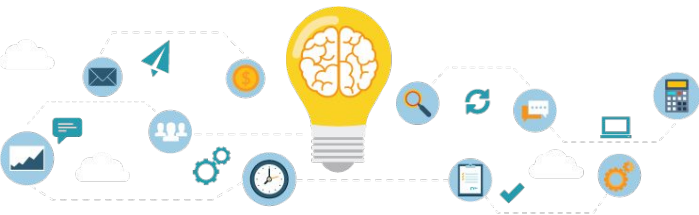
- שתי צורות לייצוא פונקציות ממודול:
 - התוצאה זהה
 - בשתיהן לא כתוב module.exports אלא רק exports
- module.exports כותבים רק כשמחזירים רפרנס לפונקציה אנונימית
- הייצוא חייב לבוא אחרי כל פונק' שרוצים לייצא ולא בסוף הקובץ כדי למנוע בעיות סנכרון בין פונקציות



Express .3

תרגיל 5 – Express routes

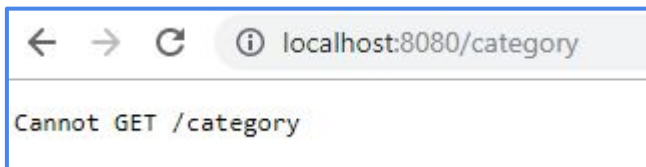
- נתרגל הרמת שרת אקספרס עם ראוטים
- הגדירו 3 נתיבים:
 - עבור עמוד הבית ב- get
 - עבור קטגוריית מוצרים (category) ב- post
 - עבור עמוד מוצר (product) ב- put
- בכל נתיב החזירו json עם שם העמוד html
- ללא התנגשות - רק אחד route צריך להתאים



תרגיל 5 - Express routes - תשובה

● בדקו: האם יכולה להיות התנגשות ?

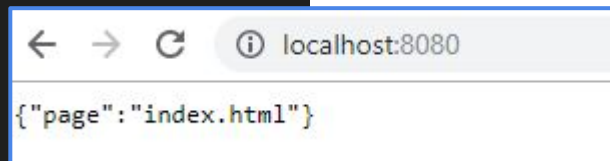
● למה זה קורה ?



● הוסיפו טיפול בראוטים ע"י (app.all) שלא מתאימים לנתיבים שהגדרנו והחזירו הודעה בהתאם.

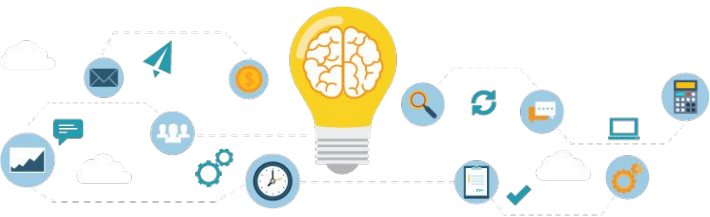
```
ex5 > JS index.js > ...
```

```
1  const express = require("express");
2  const app = express();
3  const port = process.env.PORT || 8080;
4
5  app.get("/", (req, res) => res.json({ page: "index.html" }));
6  app.post("/category", (req, res) => res.json({ page: "category.html" }));
7  app.put("/product", (req, res) => res.json({ page: "product.html" }));
8
9  app.listen(port, () =>
10 |   console.log(`Express server listening on port ${port}!`)
11 | );
```



תרגיל 6 – Express routes

- הוסיפו לקוד הקודם middleware שעבור כל נתיב 'בודק' האם המשתמש מחובר ומדפיס "Checking if the user is connected"
- עבור הנתיב /profile במתודת GET תוצג למשתמש הודעה שהוא מחובר ואח"כ שם העמוד.



תרגיל 6 - middleware - תשובה

part_7 > JS index.js > ...

```
1  const express = require('express')
2  const app = express()
3  const port = process.env.PORT || 3000
4
5  app.all('/*path}', (req, res, next) => {
6    console.log("Checking if the user is connected");
7    next();
8  });
9
10 app.get('/', (req, res) => res.json({page: 'index.html'}));
11 app.post('/category', (req, res) => res.json({page: 'category.html'}));
12 app.put('/product', (req, res) => res.json({page: 'product.html'}));
13
14 app.get("/profile", (req, res) => {
15   console.log("User is connected!");
16   res.json({page: 'profile.html'});
17 });
18
19 app.all('/*path}', (req, res) => res.json({page: '404.html'}));
20
21 app.listen(port, () => {
22   console.log(`Express server listening on port ${port}`)
23 });
```

תרגיל 7 – GET routing

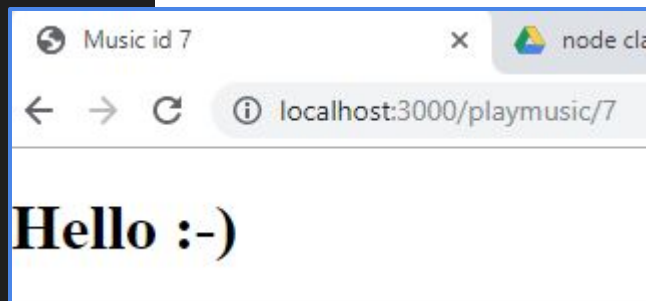
- ממשו (קוד להעתקה נמצא כאן למטה)

Route:	playmusic
Method:	GET
Request Parameters:	musicId <number>
Response:	ערך הפרמטר יופיע בכותרת הדפדפן Html
Querystring:	empty

```
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>
  <body>
    <h1>Hello :-)</h1>
  </body>
</html>
```

תרגיל 7 – GET routing

```
ex6 > JS index.js > ...
1  const express = require("express");
2  const app = express();
3  const port = process.env.PORT || 3000;
4
5  app.get("/playmusic/:musicId", (req, res) => {
6    const { musicId = null } = req.params;
7
8    console.log("musicId is", musicId);
9
10   res.send(`<!DOCTYPE html>
11     <html>
12       <head>
13         <title>Music id ${musicId}</title>
14       </head>
15       <body>
16         <h1>Hello :-)</h1>
17       </body>
18     </html>`);
19 });
20
21 app.listen(port, () =>
22   console.log(`Express server listening on port ${port}!`)
23 );
```



תרגיל 8 – POST routing

● ממשו

Route:	<code>savemusic</code>
Method:	<code>POST</code>
Request Parameters:	<code>songs[]:strings</code>
Response:	<code>json: {success: 1}</code>
Querystring:	<code>empty</code>

● הדרכה

- כדי לתמוך בקבלת פרמטרים בגוף ההודעה יש להשתמש בפונקציה `express.urlencoded` או `express.json`

תרגיל 8 – POST routing

ex8 > **JS** index.js > ...

```
1  const express = require("express");
2  const app = express();
3  const port = process.env.PORT || 3000;
4
5  app.use(express.json());
6
7  app.post("/savemusic", (req, res) => {
8    const { songs = [] } = req.body;
9
10   console.log("The songs are: ", songs);
11
12   res.json({ success: 1 });
13 });
14
15 app.listen(port, () =>
16   console.log(`Express server listening on port ${port}!`)
17 );
```

The screenshot shows a web browser's developer tools interface. At the top, the method is set to 'POST' and the URL is 'localhost:3000/savemusic'. The 'Body' tab is selected, showing the request body as a JSON object: `{ "songs": ["song1", "song2"] }`. Below this, the 'Body' tab for the response is also selected, showing the response body as a JSON object: `{ "success": 1 }`. The interface includes tabs for Params, Authorization, Headers (10), Body, Pre-request Script, Tests, and Settings. The 'Body' tab has sub-tabs for none, form-data, x-www-form-urlencoded, raw, binary, GraphQL BETA, and JSON. The 'Body' tab for the response has sub-tabs for Pretty, Raw, Preview, Visualize BETA, and JSON. The 'JSON' sub-tab is selected, showing the response body as a JSON object.